

COMP442 Assignment 1 Report

Charles-Philippe La Richeliere 40113033

January 2024

Section 1 - Lexical Specifications

The lexical specifications of the language can be described by the following regular expressions we create one for each of the different types of tokens, identifiers, integers, floats and comments. Note that the notation ϵ is for the empty string. Note that we use the following symbols for our regular expressions.

$d \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ All digits.

$d_+ \in \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ Positive digits.

$a \in \{a....z\}$ Lowercase letters.

$A \in \{A....Z\}$ Upper case letters.

The regular expressions for identifiers, integers, floats and comments.

Identifiers: $r_{id} = (a|A)(a|A|d|_+)^*$

Integers: $r_{integer} = (d_+)(d)^*|0$

Floats: $r_{float} = ((d_+)(d)^*|0).((d)^*(d_+)|0)((e|E)(+|-|\epsilon)((d_+)(d)^*|0)|\epsilon)$

The regular expressions for operators and keywords are simply the spelling of the keywords and operators so we get the regular expressions for keywords:

if, then, else, impl, func, while, for, public, private, read, write, return, self var, struct, float, void, let, inherits, integer, float

And for the operators

$=, ==, <, <=, <>, !, |, :, -, +, >, - >, /, \&, *, [,], \{, \}, (,), ., ,$

The regular expression for inline comments.

$$//(\Sigma - \text{newline})^*(\text{newline})$$

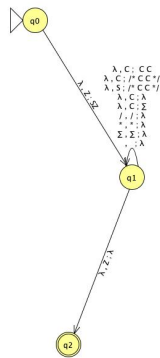
Where the Σ is the symbol for the entire alphabet of the language, and the *newline* is any symbol in the alphabet that represents a newline.

The lexical specification for block comments is not a regular language and therefore we cannot use a regular expression or represent it using an NFA or DFA. This is because we are using nested block comments this means that we must keep a count of the `"/*"` and `"*/"` encountered in the program, thus we would need to maintain a count we can represent this with a context-free grammar or a push-down automaton because the language is context-free. Thus the FSM for the lexer does not contain a FSM for the block comments.

The context-free grammar for the block comments where S is the start symbol.

$$\begin{array}{l} S \rightarrow / * CC * / \\ C \rightarrow \Sigma \\ C \rightarrow CC \\ C \rightarrow / * C * / \\ C \rightarrow \epsilon \end{array}$$

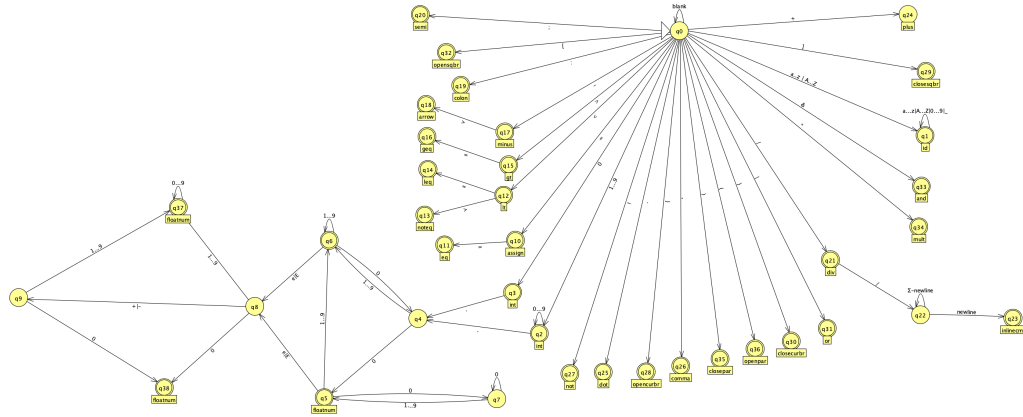
And the push-down automaton for the grammar above (obtained by converting grammar to a LL PDA using JFLAP)



To get the automaton that would represent these regular expressions we can use the conjunction of these regular expressions for the set of all valid tokens in the language.

Section 2 - Finite Automaton

This is the finite automaton for the lexical analyzer. This combines all of the regular expressions from the regular expressions that describe the lexical specification.



Note that a clearer image of the FSM is available as a jpg file in the same directory.

Section 3 - Design Overview.

We have three source files, Lexer.c, Utils.c and Lexdriver.c. The logic for the actual Lexical Analyzer and the `get_next_token()` function is in the Lexer.c file. The Utils.c file contains function definitions that are used in the Lexer.c file but are not related to the logic of the lexical analyzer.

The Lexical Analyzer is coded using C, the token is represented by the token struct, the struct has 4 member data values, these are the lexeme of the token using a character string, a syntactic category that is also a character string, a line value (integer) that is the line upon which the token was found in the text file and a valid flag that is set to 1 for a valid token and 0 for an invalid token.

The lexical analyzer was built using a hard-coded method. The lexical analyzer will read each character from the file stream one at a time, depending on the first non-blank character that it encounters it will determine the syntactic category of the next token, either an id, integer, float, keyword, operator, or

invalid character. The lexer also keeps track of the line numbers for reporting errors.

The Lexer works by appending valid characters for a certain token to the lexeme string for a token. Because we using C the string must be allocated dynamically as we will be frequently updating the token, we reallocate the memory each time that we append to the string.

For more accurate error reporting we have different classes of invalid tokens one for each different type of token. When an invalid token is encountered a token is still returned, but it is given a syntactic category of invalid and has it's valid flag set to 0.

To make sure that the token parsed is a valid token, we use a regex (POSIX from regex.h) to ensure that the token is valid token. This is an added safety measure as at the end of appending characters to the lexeme string the token should be valid.

The function takes a token pointer that has already been allocated, therefore it is the responsibility of the caller to call malloc() to allocate memory for the token and to then call free() on the lexeme of the token, then the token itself, if not the program will cause memory leaks.

Section 4 - Tools Used

The main tool that is used for the Lexical Analyzer is the C programming language using standard libraries. We use the regex.h header file to access the POSIX regular expressions library. JFLAP was used to produce the finite automaton by converting regular expressions to a finite automaton and then converting minimizing the finite automaton.